

Relatório 4 - Prática de fundamentos de TypeScript

Kaíque Medeiros Lima

1. Descrição da atividade

A entrega do card fica dividida em duas pastas. A `codigos_aula` reúne os exemplos feitos acompanhando as aulas, um arquivo por assunto, cada um executável com seu próprio `script npm demo`. A `codigos_pessoal` é um pequeno cadastro de usuários escrito do zero para aplicar os mesmos conceitos juntos, com tipos, `schema Zod`, repositório em classe e um serviço com as regras. As duas pastas usam TypeScript em modo `strict` e passam no `npm typecheck`.

a. Tipos básicos, listas e tuplas

Nos códigos da aula, no `index.ts` aparecem variáveis com `number`, `string` e `boolean`, além de listas que recebem somente valores de um tipo. Isso deixa mais claro o formato esperado dos dados e ajuda a identificar uma atribuição incompatível durante o desenvolvimento.

Também são usadas tuplas, como `[number, string]`, que definem o tipo de cada posição. A diferença para uma lista comum é que a ordem passa a fazer parte da estrutura. O exemplo de uma lista de tuplas combina essas duas ideias, permitindo guardar vários pares com o mesmo formato. Já o `any` permite trocar o tipo do valor, mas acaba abrindo mão de parte da checagem que o TypeScript oferece.

b. Uniões, enum e afirmações de tipo

Outro ponto dos exercícios é a possibilidade de aceitar mais de um tipo. O `productId` usa `string | number | boolean`, e o tipo `Grade` aceita `number` ou `string`. O símbolo `|` representa uma união, ou seja, o valor pode pertencer a uma das opções definidas, o que amplia o conjunto de valores aceitos. Isso é diferente de uma interseção, que combina as exigências de tipos diferentes e, por isso, restringe o que o objeto precisa ter.

O `enum Direction` dá nomes para valores relacionados, como `Up`, `Down` e `Left`. Também aparece uma afirmação de tipo aplicada ao `productName`. Essa sintaxe informa ao compilador como o valor deve ser tratado, mas não transforma o dado em tempo de execução. Por isso, ela precisa acompanhar o formato real do valor, em vez de servir só para esconder um aviso de tipo.

c. Objetos, tipos e interfaces

No `objects.ts`, os tipos `Order` e `User` organizam os dados de pedidos e usuários. O usuário tem campos como `nome`, `idade` e `e-mail`, uma lista de pedidos e um método `register`. A propriedade `password` aparece com `?`, mostrando que ela é opcional. Com essa estrutura, basta olhar a definição para entender o que um objeto precisa fornecer.

O arquivo também combina `Author` e `User` com `&`, formando um objeto que precisa atender aos dois tipos. Na interface `UserInterface`, o `firstName` é somente leitura, e o `readonly` impede a reatribuição dessa propriedade pelo código tipado. Já o uso de `!` em `user.password` informa ao compilador que o valor não é nulo nem indefinido, mas não faz uma verificação real; como a senha é opcional, esse cuidado continua sendo necessário na lógica.

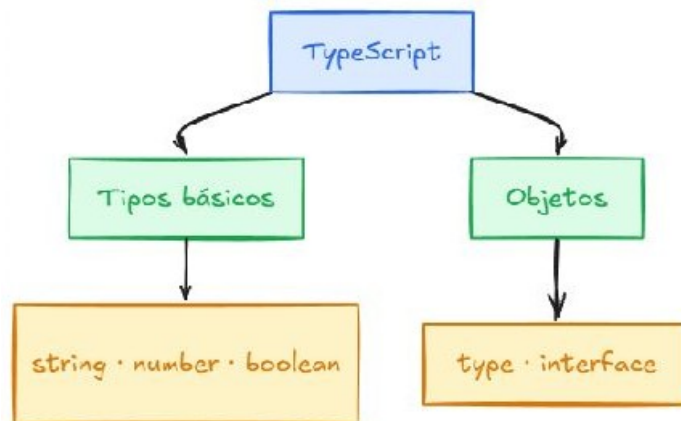


Figura 1 - Tipos básicos e estruturas de objetos. Autoria própria, Excalidraw.

d. Funções e classes

O `functions.ts` define a interface `MathFunc` para funções que recebem dois números e devolvem um número. As funções de soma e subtração seguem esse mesmo formato. Também existe uma função de log com retorno `void`, indicando que ela é usada para executar uma ação, sem devolver um resultado útil para o restante do código.

Na parte de classes, `Person` implementa `IPerson` e reúne identificador, nome, idade e o método `sayMyName`. O `id` é `readonly`, o nome é `protected` e a idade é `private`. A classe `Employee` herda de `Person` e consegue acessar o nome, mostrando o papel do `protected` na relação entre a classe base e a classe filha. O exemplo `PersonRefact` apresenta uma forma mais curta de declarar essas propriedades diretamente nos parâmetros do construtor.

e. Generics e reaproveitamento

O `generics.ts` mostra como escrever uma mesma estrutura para trabalhar com tipos diferentes. A função `returnValue` usa o parâmetro `T` tanto na entrada quanto na saída, mantendo a relação entre o tipo recebido e o devolvido. Assim, ela pode receber uma string em uma chamada e um número em outra, sem precisar virar uma função baseada em `any`.

O mesmo raciocínio aparece na função que pega o primeiro item de uma lista, na `Promise<string>` e na classe `GenericNumber`. Nessa classe, o valor inicial e a função de soma usam o tipo escolhido na criação do objeto. Isso permite reaproveitar a estrutura e ainda manter um contrato entre os valores. No caso de pegar o primeiro item, a lista vazia continua sendo uma situação que precisa de atenção, porque pode não existir um elemento para retornar.



Figura 2 - Um generic preserva a relação entre entrada e saída. Autoria própria, Excalidraw.

f. Utility types

O `utility-types.ts` parte de uma única interface `Product`, com `id`, nome, preço, descrição e estoque, e deriva os outros tipos dela em vez de reescrever os campos. `Pick` seleciona só `id`, nome e preço para um resumo, `Omit` tira o `id` para representar os dados de criação, `Partial` deixa todos os campos opcionais para uma atualização parcial e `Record` monta um catálogo

indexado por string. Se Product mudar, os tipos derivados acompanham a mudança automaticamente.

No exemplo, o patch altera apenas preço e estoque, e o catálogo usa o preço do patch quando ele existe, com o operador ?? caindo para o preço original quando não existe. O script é executado com `pnpm demo:utility`, que roda o arquivo com o `tsx`.

```
src/utility-types.ts

1 interface Product {
2   id: string;
3   name: string;
4   price: number;
5   description: string;
6   stock: number;
7 }
8
9 // Pick: pega campos
10 type ProductSummary = Pick<Product, "id" | "name" | "price">;
11
12 // Omit: tudo menos o id
13 type ProductInput = Omit<Product, "id">;
14
15 // Partial: tudo opcional
16 type ProductPatch = Partial<ProductInput>;
17
18 // Record: dicionário
19 type ProductCatalog = Record<string, ProductSummary>;
```

Figura 3 - Tipos derivados de Product com Pick, Omit, Partial e Record. Código da branch do Card 04.

```
$ pnpm demo:utility
{
  input: {
    name: 'Teclado',
    price: 150,
    description: 'Teclado mecânico ABNT2',
    stock: 10
  },
  patch: { price: 139.9, stock: 8 },
  catalog: { '1': { id: '1', name: 'Teclado', price: 139.9 } }
}
```

Figura 4 - Saída do `pnpm demo:utility`. Autoria própria, terminal.

g. Códigos pessoais: cadastro de usuários

A pasta `codigos_pessoal` junta os conceitos anteriores num domínio só. O `types.ts` usa union no `UserRole`, interface no `UserProps`, intersection no `UserWithTimestamps`, que acrescenta os campos de auditoria, e o genérico `ApiResponse<T>` para padronizar as respostas. O `collections.ts` tem a função genérica `sortBy`, que só aceita ordenar por uma chave existente do objeto, além de `Pick` no resumo do usuário e `Record` na contagem por papel. O `UserRepository` é uma classe que guarda a lista em memória, e o `userService` aplica as regras, como recusar e-mail repetido e devolver erro quando o id não existe.

A validação com `Zod` foi refeita a partir da revisão da PR. Antes, o serviço recebia objetos escritos no próprio código, que o compilador já tinha conferido, então o `parse` não

barrava nada. Agora o createUser e o updateUser recebem o body como texto, do jeito que chegaria numa requisição. O parseBody tenta o JSON.parse dentro de um try, porque um texto quebrado lança erro, e depois passa o resultado, que é unknown, pelo safeParse do schema. O tipo dos dados validados sai do próprio schema com z.infer, então regra e tipo não se separam. A requisição continua simulada com strings fixas no index.ts.

codigos_pessoal/src/schemas.ts

```
1  type ParsedBody<T> = { success: true; data: T } | { success: false; message: string };
2
3  // body de request chega como texto: JSON.parse devolve unknown e só o safeParse garante o tipo
4  export const parseBody = <S extends z.ZodType>(  
5    schema: S,  
6    body: string,  
7  ): ParsedBody<z.infer<S>> => {  
8    let payload: unknown;  
9  
10   try {  
11     payload = JSON.parse(body);  
12   } catch {  
13     return { success: false, message: "Body não é um JSON válido." };  
14   }  
15  
16   const result = schema.safeParse(payload);  
17  
18   if (!result.success) {  
19     const message = result.error.issues  
20       .map((issue) => `${issue.path.join(".")}: ${issue.message}`)  
21       .join("; ");  
22     return { success: false, message };  
23   }  
24  
25   return { success: true, data: result.data };  
26 };
```

Figura 5 - Validação do body com JSON.parse, safeParse e z.infer. Código da branch do Card 04.

No pnpm demo:users, o body válido cria o usuário u-05, o mesmo body enviado de novo é recusado por e-mail repetido, o body com nome curto, e-mail inválido e papel inexistente volta com uma mensagem por campo, e o texto que não é JSON é recusado antes de chegar ao schema. Na atualização acontece o mesmo: o campo active com texto no lugar de boolean é barrado. Na mesma revisão também saíram os tipos exportados que nenhum arquivo usava, como o UserPatch e o UpdateUserInput.

```
$ pnpm demo:users
Criação: {
  data: {
    id: 'u-05',
    name: 'Elisa Prado',
    email: 'elisa@fastcamp.dev',
    role: 'viewer',
    active: true,
    createdAt: '2026-09-23T23:30:26.596Z',
    updatedAt: '2026-09-23T23:30:26.596Z'
  },
  success: true,
  message: 'Usuário criado com sucesso.'
}
E-mail duplicado: { data: null, success: false, message: 'E-mail já cadastrado.' }
Campos inválidos: {
  data: null,
  success: false,
  message: 'name: Informe pelo menos 3 caracteres; email: Informe um e-mail válido;
  role: Papel deve ser admin, editor ou viewer'
}
JSON quebrado: { data: null, success: false, message: 'Body não é um JSON válido.' }
Atualização inválida: {
  data: null,
  success: false,
  message: 'active: Ativo deve ser true ou false'
}
Atualização de id inexistente: { data: null, success: false, message: 'Usuário não encontrado.' }
```

Figura 6 - Saída do pnpm demo:users com casos válidos e inválidos. Autoria própria, terminal.

h. Blog "Is TypeScript Frontend or Backend?"

O artigo *Is TypeScript Frontend or Backend?* apresenta o TypeScript como uma tecnologia que não fica limitada somente ao desenvolvimento frontend. Como ele é construído sobre o JavaScript, pode ser utilizado nos ambientes em que JavaScript é executado, incluindo o Node.js no backend. A principal diferença apresentada é o sistema de tipagem estática, que permite identificar incompatibilidades durante o desenvolvimento e também deixa mais explícito o formato esperado dos dados.

No frontend, o texto mostra o uso do TypeScript junto de ferramentas como Angular, React e Vue. A tipagem pode ser aplicada em propriedades, estados, parâmetros e objetos recebidos pelos componentes. Em vez de depender somente do valor recebido em tempo de execução, parte desses problemas pode ser identificada antes pelo compilador.

O artigo também apresenta o uso do TypeScript no backend, principalmente junto ao Node.js e na construção de APIs. Nesse caso, os tipos podem ser usados para definir parâmetros de funções, objetos recebidos e respostas esperadas. Também é mostrado que as duas tecnologias não precisam necessariamente competir, já que o TypeScript pode ser usado no frontend enquanto uma API em PHP fica responsável pelo backend.

2. Conclusões

Os exemplos mostram como o TypeScript ajuda a deixar os contratos do código mais visíveis. Tipos básicos organizam os valores, interfaces e tipos descrevem objetos, os utility types derivam novos formatos de um tipo existente e os generics permitem reaproveitar funções e classes sem perder essa informação. Também fica clara a diferença entre a checagem durante o desenvolvimento e as verificações que a aplicação precisa fazer quando está rodando: os tipos somem na compilação, e um dado vindo de fora, como o body de uma

requisição, só é confiável depois de passar por uma validação como o `safeParse` do `Zod`. Uma afirmação de tipo pode orientar o compilador, mas não substitui o cuidado com um dado opcional, com uma lista vazia ou com um `body` de requisição.

3. Referências

<https://www.youtube.com/watch?v=ppDsxbUNtNQ>

<https://accesto.com/blog/typescript-front-or-back-end/>

<https://zod.dev/basics>

<https://www.typescriptlang.org/docs/handbook/utility-types.html>